```
# ========================================
# GEN AI EXPERIMENT
# Zero-shot vs Few-shot vs Chain-of-Thought
# Dataset: GSM8K
# Model: TinyLlama-1.1B-Chat
# Optimized for Colab T4 GPU
# ========================================

# ================================
# STEP 1: Install Libraries
# ================================
!pip install -q transformers accelerate bitsandbytes datasets

# ================================
# STEP 2: Imports
# ================================
import torch
import pandas as pd
import re
import time
from transformers import AutoTokenizer, AutoModelForCausalLM
from datasets import load_dataset

# ================================
# STEP 3: Load GSM8K Dataset
# ================================
dataset = load_dataset("gsm8k", "main")
data = dataset["test"].to_pandas()

print("Total samples:", len(data))
print(data.head())

# ================================
# STEP 5: Load TinyLlama (8-bit)
# ================================
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
import torch

device = "cuda"

model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

# 8-bit quantization config (NEW METHOD)
bnb_config = BitsAndBytesConfig(
    load_in_8bit=True
)

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,   # ✅ NEW way
    device_map="auto"
)

model.eval()
# ================================
# STEP 6: Prompt Builder
# ================================
def build_prompt(question, mode="zero_shot", exemplars=None):

    if mode == "zero_shot":
        return (
            "Solve the following math problem.\n"
            "Give only the final numeric answer.\n\n"
            f"Question: {question}\n"
            "Answer:"
        )
```

```
    elif mode == "few_shot":
        prompt = "Here are some solved examples:\n\n"
        for q, a in exemplars:
            prompt += f"Q: {q}\nA: {a}\n\n"

        prompt += (
            "Now solve the following problem.\n"
            f"Q: {question}\n"
            "A:"
        )
        return prompt

    elif mode == "cot":
        return (
            "Solve the following math problem step by step.\n"
            "After reasoning, give the final answer in the format:\n"
            "Answer: <number>\n\n"
            f"Question: {question}\n"
            "Let's think step by step."
        )

    else:
        raise ValueError("Invalid mode")

# ================================
# STEP 7: Generate Answer
# ================================
def generate_answer(prompt, max_new_tokens=64):

    inputs = tokenizer(prompt, return_tensors="pt").to(device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            temperature=0.0,
            pad_token_id=tokenizer.eos_token_id
        )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)

# ================================
# STEP 8: Extract Numeric Answer
# ================================
def extract_last_number(text):
    numbers = re.findall(r"-?\d+\.?\d*", text)
    return numbers[-1] if numbers else None

# ================================
# STEP 9: Few-shot Examples
# ================================
def get_few_shot_examples(k=3):
    examples = []
    for i in range(k):
        q = data.iloc[i]["question"]
        a = data.iloc[i]["answer"].split("####")[-1].strip()
        examples.append((q, a))
    return examples

few_shot_examples = get_few_shot_examples(3)

# ================================
# STEP 10: Evaluation Function
# ================================
def evaluate_mode(mode, n_samples=3):

    correct = 0
    hallucinations = 0
```

```
    start_time = time.time()

    for i in range(n_samples):

        question = data.iloc[i]["question"]
        gt_answer = data.iloc[i]["answer"].split("####")[-1].strip()

        if mode == "few_shot":
            prompt = build_prompt(question, mode, few_shot_examples)
        else:
            prompt = build_prompt(question, mode)

        output = generate_answer(prompt)
        pred = extract_last_number(output)
        gt = extract_last_number(gt_answer)

        print("\n---------------------------")
        print(f"Sample {i+1}")
        print("Prediction:", pred)
        print("Ground Truth:", gt)

        if pred is None:
            hallucinations += 1
        elif pred == gt:
            correct += 1

    total_time = time.time() - start_time

    accuracy = correct / n_samples
    hallucination_rate = hallucinations / n_samples

    print("\n============================")
    print(f"Mode: {mode}")
    print(f"Accuracy: {accuracy:.3f}")
    print(f"Hallucination Rate: {hallucination_rate:.3f}")
    print(f"Inference Time: {total_time:.2f} seconds")
    print("============================\n")

    return accuracy, hallucination_rate, total_time

# ================================
# STEP 11: Run Experiment
# ================================
acc_zero, hall_zero, time_zero = evaluate_mode("zero_shot")
acc_few, hall_few, time_few = evaluate_mode("few_shot")
acc_cot, hall_cot, time_cot = evaluate_mode("cot")

# ================================
# STEP 12: Final Comparison
# ================================
print("FINAL RESULTS")
print("---------------------------------")
print(f"Zero-shot  | Acc: {acc_zero:.3f} | Hall: {hall_zero:.3f} | Time: {time_zero:.2f}s")
print(f"Few-shot   | Acc: {acc_few:.3f} | Hall: {hall_few:.3f} | Time: {time_few:.2f}s")
print(f"CoT        | Acc: {acc_cot:.3f} | Hall: {hall_cot:.3f} | Time: {time_cot:.2f}s")
print("---------------------------------")
```